

Работа с SourceCraft Security API

Обновлено 16 сентября 2026, 10:55 Денис Макрушин

SourceCraft Security API позволяет получать результаты сканирований, обрабатывать группы дефектов и находки, загружать результаты внешних анализаторов и формировать отчёты.

На этой странице описаны основные сценарии для **SCS Backend API 1.0.0**. Полный перечень методов, параметров и схем — в [Swagger UI](#); основа этой версии страницы — [предоставленная выгрузка OpenAPI](#). Обзор инструментов приведён в разделе [Безопасность в SourceCraft](#).

Подготовьте доступ

Уточните базовый адрес Security API, способ аутентификации и необходимые права у администратора или владельца API: в предоставленной спецификации они не указаны. Это не означает, что запросы доступны без авторизации.

Все методы требуют query-параметр `gitRepo` — внутренний ID репозитория, переданный строкой, или его публичный UUID. В описании указаны поля `id` и `uuid` ответа `getRepo`; сам метод получения репозитория в эту выгрузку не входит. Название, slug и Git URL вместо идентификатора не описаны.

Для работы с результатами различайте идентификаторы:

Идентификатор	Назначение
<code>scanUuid</code>	UUID сканирования.
<code>publicId</code> / <code>defectGroupPublicId</code>	Номер группы дефектов <code>#N</code> внутри репозитория. Используется для получения группы, истории и изменения статуса.
<code>defectGroupUuid</code>	UUID группы. Используется для получения находок и добавления связей.

Публичный номер и UUID группы доступны в `DefectGroupDto`; они не взаимозаменяемы. Внутренний числовой `id` группы помечен устаревшим.

Получите результаты сканирования

Начните с последнего сканирования. В примере замените значения в угловых скобках. `APPSEC_BASE_URL` — адрес API без `/v1`, `APPSEC_CURL_CONFIG` — локальный файл только с согласованными параметрами аутентификации cURL. Не храните его в репозитории. Это обозначения примера, а не предопределённые переменные SourceCraft.

```
export APPSEC_BASE_URL='https://<адрес-Security-API>'
export APPSEC_CURL_CONFIG='<полный-путь-к-файлу-аутентификации-cURL>'
export GIT_REPO='<UUID-репозитория>'

curl --disable --config "$APPSEC_CURL_CONFIG" \
  --fail-with-body --silent --show-error \
  --get "${APPSEC_BASE_URL%}/v1/scans/latest" \
  --data-urlencode "gitRepo=$GIT_REPO"
```

Ответ `200` содержит `ScanSummaryDto`. Используйте его `uuid` как `scanUuid` для получения подробностей и результатов конкретного сканирования.

Задача	Метод	Основные параметры помимо <code>gitRepo</code>
История сканирований	<code>GET /v1/scans</code>	<code>status</code> , <code>pageSize</code> , <code>pageToken</code>
Подробности сканирования	<code>GET /v1/scans/{scanUuid}</code>	<code>scanUuid</code> в пути
Группы дефектов	<code>GET /v1/defect-groups</code>	Выбор сканирования, фильтры и пагинация — см. ниже
Отдельная группа	<code>GET /v1/defect-groups/{publicId}</code>	Публичный номер в пути
Находки группы	<code>GET /v1/findings</code>	<code>defectGroupUuid</code> , <code>pageSize</code> , <code>pageToken</code>

Выбор результатов. У списка сканирований значение `status` по умолчанию — `FINISHED`, а не все состояния. Для групп дефектов передайте `scanUuid`, чтобы выбрать конкретное сканирование. Без него по описанию используется последний запуск выбранного типа: `SCAN_TYPE_DEFAULT` — CI, `SCAN_TYPE_AUDIT` — аудит. При заданном `scanUuid` параметр `scanType` игнорируется.

Фильтры групп. Доступны `type`, `severity`, `status`, `rule`, `description` и `file`. Несколько статусов или уровней критичности передавайте повторением ключа, например `severity=HIGH&severity=CRITICAL`. Допустимые значения смотрите в схеме конкретного метода: перечни типов движков в разных операциях отличаются.

Пагинация. Списки возвращают `data`, `nextPageToken` и `totalSize`. `pageSize` — от 1 до 250, по умолчанию 50. В запросе первой страницы явно задайте `pageSize=50` и `pageToken=`, для следующей передайте `nextPageToken` из ответа как `pageToken`. Признак окончания выдачи в выгрузке не определён.

В ряде методов текст разрешает пропустить параметр, но схема помечает его обязательным. Не считайте приведённую таблицу полным списком обязательных полей: проверяйте схему операции. Числовые `status` в кратком ответе сканирования и `status / severity` в группе дефектов не имеют таблицы соответствия; не определяйте их значения по порядку строковых enum.

Обработайте группы дефектов

Изменение статуса. Сначала запросите `GET /v1/defect-groups/{publicId}/allowed-transitions` с `gitRepo`. Выберите целевой статус из `allowedTransitions`, затем передайте JSON в `POST /v1/defect-groups/status/batch?gitRepo=...`.

Пример тела: замените `42` номером группы; `RESOLVED_FP` используйте только при разрешённом переходе и подтверждённом ложном срабатывании.

```
{
  "updates": [{ "publicId": 42, "status": "RESOLVED_FP" }],
  "comment": "Проверено: ложное срабатывание"
}
```

Ответ содержит `updated` — количество изменённых групп. Пакет должен содержать минимум один элемент; необязательный комментарий ограничен 1023 символами. Атомарность пакета не описана. Установка статуса не является документированным способом запуска ИИ-триажа или исправления.

История и комментарии. `GET /v1/activities` принимает `gitRepo`, `defectGroupPublicId` и параметры пагинации. Для комментария используйте `POST /v1/activities/comment` с `gitRepo`: в JSON передайте ровно один из `defectGroupPublicId / defectGroupUuid` и `content` с непробельным текстом длиной до 1023 символов. Изменение и удаление комментариев по `activityUuid` описаны в Swagger; успешное удаление возвращает `204` без тела.

Связи с задачами. `POST /v1/defect-groups/links/batch` добавляет связи по UUID группы. Для `SOURCECRAFT_ISSUE` и `SOURCECRAFT_PR` передавайте ID платформы строкой; для `EXTERNAL_URL` и `YA_TRACKER` — HTTPS URL. Поле `inserted` считает добавленные связи: уже существующие связи и связи для групп, отсутствующих в репозитории, пропускаются. Обратный поиск доступен через `GET /v1/defect-groups/by-link`. Полные тела и параметры, а также отдельный метод `ticket-url/batch` приведены в Swagger.

Загрузите отчёт внешнего анализатора

Используйте `POST /v1/reports/upload?gitRepo=...` с телом `multipart/form-data`. Обязательные поля: `file`, `commit` — полный SHA проверенного коммита, `engineName` и `reportType`. Содержимое файла должно соответствовать формату: например, SARIF для `reportType=SARIF`, SPDX или CycloneDX JSON для `SBOM`.

Пример синхронной загрузки SARIF для SAST использует переменные из раздела выше. Замените SHA на коммит, для которого сформирован `result.sarif`.

```
COMMIT_SHA='<полный-SHA-проверенного-коммита>'

curl --disable --config "$APPSEC_CURL_CONFIG" \
  --fail-with-body --silent --show-error \
  --request POST "${APPSEC_BASE_URL%}/v1/reports/upload?gitRepo=$GIT_REPO" \
  --form 'file=@result.sarif' \
  --form-string "commit=$COMMIT_SHA" \
  --form-string 'engineName=my-security-analyzer' \
  --form-string 'reportType=SARIF' \
  --form-string 'engineType=SAST' \
  --form-string 'isReactive=false' \
  --form-string 'shouldTriggerEnrichment=false'
```

`SAST` и отключённое обогащение выбраны для примера, а не заданы как значения по умолчанию. Остальные поля формы описаны в Swagger. Заголовок `multipart` вручную не задавайте.

Ограничения файла. Тело запроса — не более **100 MB**. Можно передать ZIP с ровно одним файлом в корне, без каталогов и дополнительных файлов; размер этого файла после распаковки также не должен превышать **100 MB**.

Результат обработки. При `isReactive=false` ответ `200` означает успешную обработку, а `400` — некорректный запрос или отчёт. При `isReactive=true` ответ `202` означает только приём до разбора содержимого: последующая ошибка отражается в логах. Метода проверки фонового импорта в выгрузке нет. Для `AUDIT` описание рекомендует синхронный режим, чтобы получить причину отклонения.

Для загрузки также описаны `401` — проблема с токеном, `403` — недостаточные права или отключённая безопасность репозитория, `404` — репозиторий не найден.

Сформируйте и скачайте PDF или SARIF

Генерация отчёта и загрузка внешнего результата — разные операции. Для генерации выполните последовательность:

Шаг	Запрос	Что проверить
Запустить генерацию	<code>POST /v1/reports/generate</code>	Передайте query-параметры <code>gitRepo</code> , <code>scanUuid</code> , <code>format</code> , <code>engineType</code> . Ответ <code>200</code> содержит операцию, а не готовый файл.
Проверить состояние	<code>GET /v1/reports/operations/{operationUuid}</code> с <code>gitRepo</code>	При <code>IN_PROGRESS</code> проверьте позднее. Продолжайте только при <code>done=true</code> и <code>status=COMPLETED</code> ; при <code>FAILED</code> не скачивайте результат.
Получить ссылку	<code>GET /v1/reports/presigned-url</code>	Для выбранной операции передайте <code>gitRepo</code> , полученный <code>reportUuid</code> , соответствующие <code>scanUuid</code> , <code>format</code> , <code>engineType</code> . Ответ содержит <code>presignedUrl</code> .
Скачать файл	Запрос по <code>presignedUrl</code>	Не передавайте на адрес хранилища токен или файл аутентификации Security API. Не публикуйте подписанную ссылку.

`format` — PDF или SARIF. Для SARIF `engineType=UNSPECIFIED` выбирает объединённый отчёт; доступны также `SECRETS`, `SCA`, `SAST`, `DAST`. Для PDF тип движка игнорируется. Полные правила выбора последнего сканирования и поиска отчёта — в описаниях методов; в последовательности выше параметры заданы явно из-за расхождений их обязательности в схеме.

Если отчёт уже существует, API возвращает завершённую операцию без повторной генерации; если создаётся — текущую операцию. Срок действия ссылки и интервал опроса не указаны. Метод проверки операции относится **только к генерации**, не к фоновому импорту.

Дополнительные методы и ограничения

Настройки безопасности репозитория читаются и изменяются через `GET / POST /v1/admin/repository-statuses`. Они выделены в административную группу; права необходимо согласовать отдельно. Это изменение настроек, а не команда запуска сканирования. Подробный контракт настроек — в Swagger.

В этой выгрузке нет отдельных методов запуска или отмены сканирования, ИИ-аудита, триажа и автоматического исправления. Наличие соответствующих полей или значений `enum` не заменяет описание таких операций.

Общий формат ошибок, лимиты частоты обращений и гарантии безопасного повтора изменяющих запросов не определены. Не трактуйте ошибку запроса как отсутствие уязвимостей; после тайм-аута изменения сначала проверьте состояние. Примеры сверены со спецификацией, но не проверены на рабочем сервисе.